



Support Vector Machines (SVM)

Margen máximo, regularización y el método de kernels

Círculo de Ciencia de Datos y Econometría

2 de septiembre de 2026

Mapa de la presentación

1. Geometría y margen máximo
2. Soft margin y regularización
3. Dualidad y kernels
4. Hiperparámetros y uso práctico
5. Aplicación en Python
6. Cierre y fuentes

Pregunta guía

¿Cómo construye una SVM una frontera con buena capacidad de generalización y cómo puede volverla no lineal sin trabajar explícitamente en un espacio enorme?

Fuentes troncales

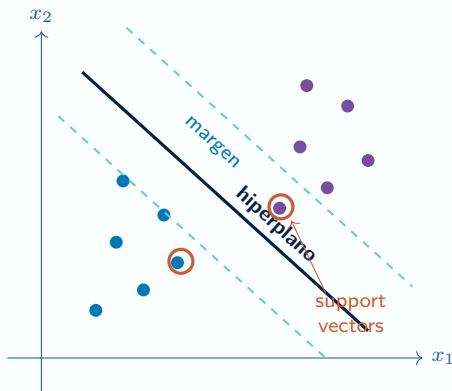
Géron, cap. 5; James et al., cap. 9; Hastie et al., cap. 12.

Aplicación

Python + scikit-learn, código reproducible.

¿Qué es una Support Vector Machine?

- ▶ Es un método supervisado para **clasificación**, regresión y detección de novedades.
- ▶ En clasificación busca una frontera que separe clases con el **mayor margen posible**.
- ▶ La frontera queda determinada por unas pocas observaciones críticas: los **support vectors**.
- ▶ Con kernels puede construir fronteras no lineales manteniendo un problema de optimización convexo.



Fuente: Géron (2022), cap. 5; James et al. (2013), cap. 9.

Hiperplano, función de decisión y distancia

Hiperplano en $\mathbb{R}^p$

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

Regla de clasificación

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

Distancia firmada al hiperplano

$$\frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|}$$

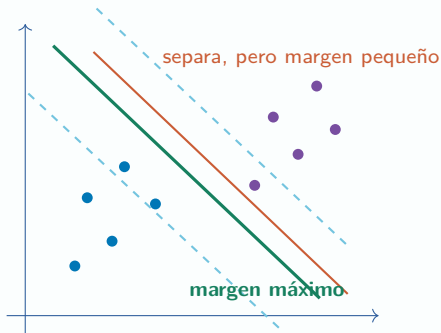
Lectura geométrica

- ▶ $\mathbf{w}$ es normal al hiperplano.
- ▶ b desplaza la frontera.
- ▶ Escalar $(\mathbf{w}, b)$ por la misma constante no cambia la frontera.
- ▶ Por eso fijamos una escala conveniente para expresar el margen.

La SVM usa esta libertad de escala para imponer que los puntos más cercanos satisfagan $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$.

Fuente: James et al. (2013), secs. 9.1.1–9.1.4; Hastie et al. (2009), cap. 12.

¿Por qué maximizar el margen?



- ▶ Si varias fronteras clasifican igual al conjunto de entrenamiento, necesitamos un criterio para elegir una.
- ▶ La SVM elige el hiperplano que deja la **calle más ancha** entre las clases.
- ▶ Con la normalización canónica, la distancia desde la frontera a cada borde del margen es $1/\|\mathbf{w}\|$.

$$\text{Ancho total: } \frac{2}{\|\mathbf{w}\|} \implies \text{minimizar } \|\mathbf{w}\|^2.$$

Fuente: James et al. (2013), sec. 9.1.3; Géron (2022), "Large Margin Classification".

Hard margin: problema de optimización

SVM lineal de margen duro

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2$$

$$\text{s.a.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n$$

Todos los puntos deben quedar del lado correcto y fuera de la franja del margen. No se permite ninguna violación.

¿Qué consigue?

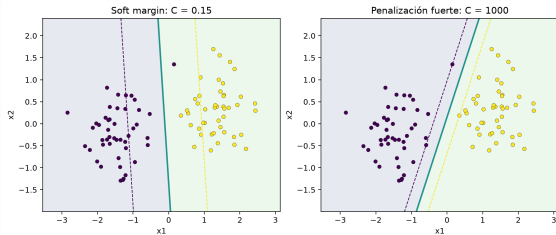
- ▶ Separación perfecta, si existe.
- ▶ Solución de un problema cuadrático convexo.
- ▶ Óptimo global, no un mínimo local arbitrario.

Condición fuerte

Debe existir un hiperplano que separe completamente las clases.

Fuente: Hastie et al. (2009), cap. 12; Géron (2022), apéndice matemático de SVM.

Por qué el hard margin suele ser demasiado rígido



Problema 1: separabilidad

Si las clases se solapan, el conjunto de restricciones del hard margin puede ser imposible.

Problema 2: outliers

Una sola observación extrema puede mover drásticamente la frontera para conservar separación perfecta.

Necesitamos permitir errores controlados: eso conduce al **soft margin**.

Fuente: Géron (2022), "Soft Margin Classification". Figura generada por `codigo_demo_svm.py`.

Soft margin: permitir violaciones con variables de holgura

Formulación primal usada por scikit-learn

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.a.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

ξ_i mide cuánto puede violar el margen la observación i ; si cruza la frontera, la violación es aún mayor.

Dos objetivos en tensión

1. Margen grande: minimizar $\|\mathbf{w}\|^2$.
2. Pocas violaciones: minimizar $\sum_i \xi_i$.

El papel de C

C decide cuánto penalizamos las violaciones frente a la anchura del margen.

Fuente: Géron (2022), "Soft Margin Classification" y Eq. 5-2.

C controla el trade-off sesgo–varianza

En scikit-learn: C pequeño

- ▶ Más regularización.
- ▶ Margen más ancho.
- ▶ Tolera más violaciones.
- ▶ Menor varianza, más riesgo de underfitting.

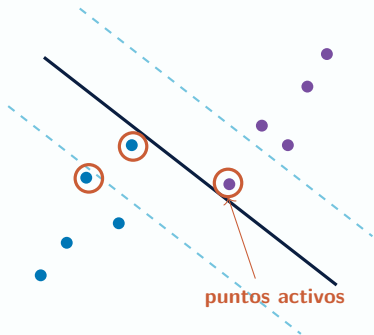
En scikit-learn: C grande

- ▶ Menos regularización.
- ▶ Penaliza fuertemente violaciones.
- ▶ Frontera más ajustada a la muestra.
- ▶ Menor sesgo, más riesgo de overfitting.

Cuidado con la notación: ISLR (James et al.) usa otra parametrización de C como presupuesto de violaciones, por lo que allí un C mayor implica un margen más ancho. En scikit-learn y en Géron, C es la penalización de las violaciones y la relación es la contraria.

Fuente: Géron (2022), cap. 5; James et al. (2013), sec. 9.2.2.

Support vectors: los puntos que realmente fijan la frontera



- ▶ Son observaciones sobre el margen, dentro del margen o mal clasificadas.
- ▶ En la solución dual tienen multiplicadores $\alpha_i > 0$.
- ▶ Observaciones suficientemente alejadas del margen tienen $\alpha_i = 0$.
- ▶ Por ello la función de decisión depende de un subconjunto de entrenamiento.

Esta **esparsidad** es una propiedad importante: los puntos lejanos a la frontera no cambian la solución directamente.

Fuente: James et al. (2013), secs. 9.1.3 y 9.2; Hastie et al. (2009), cap. 12.

La formulación dual revela dónde entra el kernel

Problema dual del soft-margin SVM

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \underbrace{\mathbf{x}_i^\top \mathbf{x}_j}_{\text{producto interno}} \\ \text{s.a.} \quad & 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

La optimización ya no necesita las coordenadas por separado: solo aparecen productos internos entre pares de observaciones.

Consecuencia

Si reemplazamos

$$\mathbf{x}_i^\top \mathbf{x}_j \quad \text{por} \quad K(\mathbf{x}_i, \mathbf{x}_j),$$

obtenemos un clasificador lineal en otro espacio de características sin tener que construirlo explícitamente.

Fuente: Hastie et al. (2009), cap. 12; Géron (2022), “The Dual Problem” y “Kernelized SVMs”.

Cambio de dimensionalidad: una frontera no lineal puede volverse lineal

Espacio original: 1D

clase +1 en ambos extremos



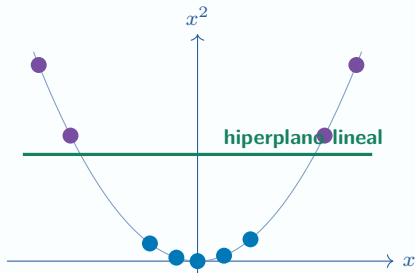
clase -1 en el centro

$$\phi(x) = (x, x^2)$$

No separable con un umbral

Una sola frontera lineal en 1D es un punto; no puede dejar +1 en ambos extremos y -1 en el centro.

Espacio transformado: 2D



Una frontera lineal en el espacio ampliado se proyecta como una frontera **no lineal** en el espacio original.

Fuente: Intuición de expansión de bases: James et al. (2013), sec. 9.3; Hastie et al. (2009), secs. 12.3 y 12.4.

El kernel trick: calcular similitudes sin construir $\phi(x)$

Supongamos una transformación potencialmente enorme:

$$\phi : \mathbb{R}^p \rightarrow \mathcal{H}$$

Un kernel válido satisface

$$K(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle_{\mathcal{H}}$$

La SVM solo necesita esos productos internos.

Por qué es potente

- ▶ Un kernel polinómico equivale a usar interacciones y potencias.
- ▶ El RBF corresponde a un espacio de características implícito de dimensión infinita.
- ▶ El costo depende de evaluar K , no de materializar ese espacio.

No calculamos todas las nuevas variables. Construimos la matriz de Gram $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ y optimizamos con ella.

Condición matemática

Para kernels estándar, la matriz de Gram debe ser simétrica y semidefinida positiva; así puede interpretarse como un producto interno en algún espacio de Hilbert.

Fuente: James et al. (2013), sec. 9.3; Hastie et al. (2009), secs. 12.3.3 y 12.3.7.

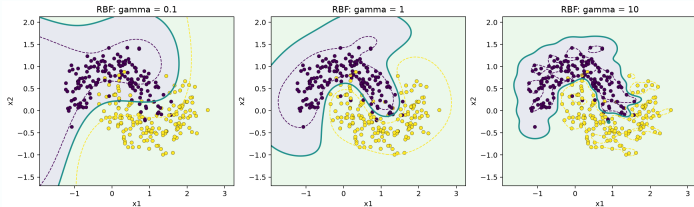
Kernels más usados

Kernel	Fórmula	Intuición / hiperparámetros
Lineal	$K(x, z) = x^\top z$	Sin expansión no lineal; suele funcionar bien cuando p es grande.
Polinómico	$K(x, z) = (\gamma x^\top z + r)^d$	d controla grado; γ escala la similitud; $r = \text{coef0}$ cambia peso entre términos de distinto orden.
RBF / Gaussiano	$K(x, z) = \exp(-\gamma \ x - z\ ^2)$	Similitud local. γ alto: influencia muy local; γ bajo: frontera más suave.
Sigmoide	$K(x, z) = \tanh(\gamma x^\top z + r)$	Menos común; se parece formalmente a una neurona sigmoide.

En práctica, la comparación más importante suele ser **lineal vs. RBF**; los hiperparámetros deben elegirse con validación cruzada, no por ajuste visual al training set.

Fuente: James et al. (2013), Eqs. 9.22–9.24; Géron (2022), “Polynomial Kernel” y “Gaussian RBF Kernel”.

RBF en profundidad: γ define el radio de influencia



γ bajo

La similitud cae lentamente con la distancia. Cada punto influye sobre una región grande: frontera suave, mayor sesgo.

γ alto

La similitud cae muy rápido. Cada punto influye localmente: frontera ondulada, mayor varianza.

Fuente: Géron (2022), Figura 5-9; James et al. (2013), Eq. 9.24. Figura generada localmente.

La función de decisión kernelizada

Después de resolver el dual:

$$f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

$$\hat{y} = \text{sign}(f(\mathbf{x}))$$

Solo aparecen los support vectors: aquellos con $\alpha_i > 0$. Para un RBF, los support vectors cercanos a x reciben mayor similitud y pesan más en la decisión.

Lectura operacional

Para predecir una nueva observación:

1. medir su similitud con cada support vector;
2. ponderar por $\alpha_i y_i$;
3. sumar el intercepto b ;
4. tomar el signo.

Fuente: James et al. (2013), Eq. 9.23; Hastie et al. (2009), cap. 12.

Hiperparámetros principales en `sklearn.svm.SVC`

Parámetro	Actúa sobre	Qué ocurre al modificarlo
<code>C</code>	Regularización / penalización	Mayor C : menos tolerancia a violaciones; menor C : margen más ancho y más regularización.
<code>kernel</code>	Geometría de la frontera	<code>linear</code> , <code>rbf</code> , <code>poly</code> , <code>sigmoid</code> o función propia.
<code>gamma</code>	Escala del RBF/poly/sigmoid	Alto: influencia local y mayor flexibilidad. Bajo: frontera más suave. <code>scale</code> usa una regla basada en varianza.
<code>degree</code>	Kernel polinómico	Grado d ; solo importa con <code>poly</code> .
<code>coef0</code>	Poly / sigmoid	Término independiente r ; altera peso relativo de términos de orden alto y bajo.
<code>class_weight</code>	Costos por clase	Útil con desbalance: permite penalizar más errores de una clase.

Fuente: Géron (2022), cap. 5; API conceptual de SVC usada en el código asociado.

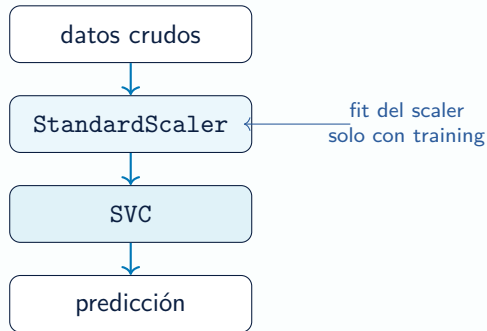
Escalar variables no es opcional en SVM

Sin escalamiento

Si una variable está en miles y otra entre 0 y 1, las distancias y productos internos quedan dominados por la primera.

Con StandardScaler

Las variables se centran y escalan antes de estimar la SVM. Para evitar fuga de información, el escalamiento debe vivir dentro de un Pipeline.



Fuente: Géron (2022), Figura 5-2 y ejemplo de pipeline con StandardScaler.

Cómo ajustar C y γ sin engañarnos

Flujo recomendado

1. Separar train y test.
2. Pipeline: escalamiento + SVC.
3. Validación cruzada dentro de train.
4. Explorar escala logarítmica para C y γ .
5. Elegir por una métrica coherente con el problema.
6. Evaluar una sola vez en test.

Ejemplo de grilla

$$C \in \{0,1, 1, 10, 100\}$$

$$\gamma \in \{0,1, 1, 10, \text{scale}\}$$

En clasificación desbalanceada, accuracy puede ser pobre criterio. Considerar ROC-AUC, PR-AUC, recall, precision y costos de error.

Fuente: Géron (2022), selección de modelos y cap. 5; James et al. (2013), sec. 9.6.2.

Ventajas y desventajas

Ventajas

- ▶ Excelente capacidad predictiva en datos pequeños/medianos.
- ▶ Muy útil en alta dimensión, incluso con $p \gg n$.
- ▶ Kernels capturan fronteras no lineales complejas.
- ▶ Problema convexo: solución global.
- ▶ Solo support vectors fijan la frontera.
- ▶ Regularización explícita mediante C .

Desventajas

- ▶ No escala bien a conjuntos muy grandes, especialmente SVC kernelizado.
- ▶ Sensible al escalamiento de variables.
- ▶ C y γ pueden exigir búsqueda cuidadosa.
- ▶ Menos interpretable que modelos lineales o árboles simples.
- ▶ Predicciones probabilísticas no son nativas.
- ▶ Elegir un kernel inadecuado puede aumentar mucho sesgo o varianza.

Fuente: Géron (2022), introducción del cap. 5; James et al. (2013), cap. 9.

¿Cuándo elegir una SVM?

Alta dimensión

Texto, genómica, muchas variables respecto al número de observaciones. Un kernel lineal puede ser suficiente.

Frontera no lineal

Cuando un modelo lineal falla y el tamaño muestral permite trabajar con RBF o polinómico.

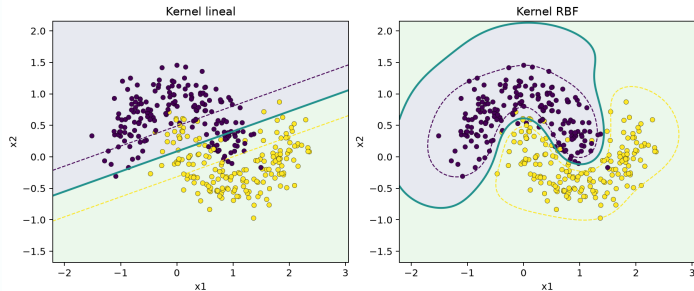
Precisión > explicación

Cuando importa más una frontera robusta que una interpretación directa de coeficientes.

No es primera opción para millones de observaciones con kernel RBF. En ese caso conviene comparar con modelos lineales escalables, árboles/boosting o aproximaciones al kernel.

Fuente: Géron (2022), cap. 5; James et al. (2013), aplicación de expresión génica en sec. 9.6.5.

Ejemplo visual: lineal vs. RBF sobre una frontera curva



Kernel lineal

Solo puede construir un hiperplano en el espacio original. Con `make_moons` queda sesgado.

Kernel RBF

Construye una separación curvada mediante similitudes locales en el espacio implícito.

La flexibilidad adicional debe validarse fuera de muestra.

Fuente: Figura generada por `codigo_demo_svm.py`, semilla 42.

Python 1/3: pipeline y separación train/test

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```
X, y = make_moons(
    n_samples=600, noise=0.25, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30,
    stratify=y, random_state=42
)

model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=10, gamma="scale")
)
model.fit(X_train, y_train)
```

Puntos clave

- ▶ Escalamiento dentro del pipeline.
- ▶ stratify conserva proporciones.
- ▶ Semilla fija para reproducibilidad.
- ▶ RBF para una frontera no lineal.

Python 2/3: validación cruzada de C , γ y kernel

```
from sklearn.model_selection import GridSearchCV

pipe = make_pipeline(StandardScaler(), SVC())
param_grid = [
    {
        "svc__kernel": ["linear"],
        "svc__C": [0.1, 1, 10, 100],
    },
    {
        "svc__kernel": ["rbf"],
        "svc__C": [0.1, 1, 10, 100],
        "svc__gamma": ["scale", 0.1, 1, 10],
    },
]

search = GridSearchCV(
    pipe, param_grid, scoring="roc_auc",
    cv=5, n_jobs=-1, refit=True
)
search.fit(X_train, y_train)
```

Por qué así

- ▶ La grilla vive solo en training.
- ▶ Escalamiento se re-estima en cada fold.
- ▶ Se compara lineal vs. RBF.
- ▶ ROC-AUC guía la selección.

Python 3/3: evaluación fuera de muestra

```
from sklearn.metrics import (
    accuracy_score, confusion_matrix, roc_auc_score
)

best = search.best_estimator_
pred = best.predict(X_test)
score = best.decision_function(X_test)

print(search.best_params_)
print("accuracy =", accuracy_score(y_test, pred))
print("roc_auc =", roc_auc_score(y_test, score))
print(confusion_matrix(y_test, pred))
print(
    "support vectors =",
    best.named_steps["svc"].n_support_
)
```

Resultado reproducible

Mejor grilla: RBF, $C = 10$,
 $\gamma = \text{scale}$.

ROC-AUC CV: **0.9724**.

Accuracy test: **0.9167**.

ROC-AUC test: **0.9817**.

Support vectors: $42 + 39 = 81$.

Fuente: Ejecución local de `codigo_demo_svm.py`; scikit-learn, semilla 42.

Lineal vs. RBF: resultado del mismo experimento

Modelo	Accuracy	ROC-AUC	SV
SVM lineal	0.8556	0.9383	141
SVM RBF	0.9333	0.9775	75

Lectura

- ▶ La estructura de `make_moons` es no lineal.
- ▶ El RBF reduce el sesgo respecto al hiperplano lineal.
- ▶ Menos support vectors no implica siempre mejor modelo, pero aquí la frontera RBF es además más parsimoniosa en puntos activos.

Fuente: Ejecución de `codigo_demo_svm.py`, semilla 42.

Ideas que deben quedar

1. Una SVM lineal busca el hiperplano con máximo margen.
2. Hard margin exige separación perfecta; soft margin introduce holguras y regularización.
3. En scikit-learn, C alto penaliza más los errores; C bajo regulariza más.
4. La formulación dual depende de productos internos, lo que habilita kernels.
5. El kernel trick equivale a trabajar en un espacio de características ampliado sin construirlo explícitamente.
6. En RBF, γ gobierna la localidad; C y γ deben validarse conjuntamente, siempre dentro de un pipeline con escalamiento y validación cruzada.

Margen
+
Regularización
+
Kernel
⇓
Fronteras flexibles con optimización convexa

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3.^a ed., O'Reilly.

Capítulo 5: clasificación SVM lineal, soft margin, kernels, RBF, complejidad y formulación matemática.

James, G.; Witten, D.; Hastie, T.; Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.

Capítulo 9: maximal margin classifier, support vector classifier, kernels, RBF, multiclase y laboratorio.

Hastie, T.; Tibshirani, R.; Friedman, J. (2009). *The Elements of Statistical Learning*, 2.^a ed., Springer.

Capítulo 12: formulación avanzada de SVM, penalización, reproducing kernels y perspectiva estadística.

Material reproducible y compilación

Código asociado

`codigo_demo_svm.py`

Genera las métricas y las figuras PNG utilizadas por la presentación.

```
pip install numpy matplotlib  
scikit-learn  
python codigo_demo_svm.py
```

Presentación Beamer

`presentacion-svm.tex`

```
pdflatex presentacion-svm.tex  
pdflatex presentacion-svm.tex
```

El logo se reutiliza desde la carpeta Árboles de Decisión; si no está disponible, el documento compila con un fallback textual.